# Section 1: Docker Fundamentals

Docker is a platform for building, shipping, and running applications inside lightweight, isolated environments called containers. Unlike virtual machines, containers share the host operating system's kernel, which makes them start much faster and use fewer resources than a full VM.

## Images and Containers

A Docker image is a read-only template that contains everything needed to run an application: the code, runtime, libraries, and system tools. When you run an image, Docker creates a container, which is a live, running instance of that image. Multiple containers can be started from the same image, each running independently.

## Volumes and Persistence

Docker volumes are the recommended mechanism for persisting data generated by containers, because container filesystems are ephemeral and are wiped when the container is removed. Docker volumes are managed entirely by Docker itself and live outside the container's writable layer, typically at a path such as /var/lib/docker/volumes on the host machine.

## Networking

By default, Docker creates a bridge network for containers on a single host, allowing them to communicate with each other via internal IP addresses. For multi-host communication, Docker supports an overlay network driver, which is commonly used with Docker Swarm or in combination with orchestrators like Kubernetes.

# Section 2: Kubernetes Basics

Kubernetes is a container orchestration platform that automates the deployment, scaling, and management of containerized applications across a cluster of machines.

## Pods

The smallest deployable unit in Kubernetes is a Pod, which can contain one or more tightly coupled containers that share the same network namespace and storage volumes. Containers within a single Pod can communicate with each other over localhost.

## Self-Healing

Kubernetes continuously monitors the health of Pods using liveness and readiness probes. If a container fails its liveness probe, Kubernetes automatically restarts it. The default restart backoff delay starts at 10 seconds and doubles with each subsequent failure, up to a cap of 5 minutes.

## Services

A Kubernetes Service provides a stable network endpoint for a set of Pods, since individual Pods are ephemeral and can be recreated with new IP addresses at any time. The most common Service type for internal cluster communication is ClusterIP.

# Section 3: Distributed System Design

A distributed system is a collection of independent computers that appear to users as a single coherent system. The primary challenges are network partitions, partial failures, and maintaining consistency across nodes.

## The CAP Theorem

The CAP theorem states that a distributed data store can only guarantee two out of three properties at the same time: Consistency, Availability, and Partition tolerance. Since network partitions are unavoidable in real systems, most designs choose to trade off between consistency and availability during a partition event.

## Load Balancing

Load balancers distribute incoming requests across multiple servers to prevent any single server from becoming a bottleneck. Common strategies include round robin, least connections, and consistent hashing, with consistent hashing being particularly useful for minimizing cache invalidation when servers are added or removed.

## Idempotency

An idempotent operation produces the same result no matter how many times it is executed. This property is critical in distributed systems because network retries are common, and a non-idempotent operation retried after a timeout can cause duplicate side effects, such as charging a customer's card twice for one purchase.

# Section 4: React Fundamentals

React is a JavaScript library for building user interfaces using a component-based architecture. Components can hold their own state using the useState hook, and can run side effects such as data fetching using the useEffect hook.

## The Virtual DOM

React uses a virtual DOM to minimize expensive direct manipulations of the real browser DOM. When state changes, React computes the difference between the previous and new virtual DOM trees and applies only the minimal set of real DOM updates needed.

## Memoization Pitfalls

A commonly overlooked detail: passing an inline function or object as a prop to a child component on every render will cause that child to re-render unnecessarily, even if wrapped in React.memo, unless the function or object reference itself is memoized using useCallback or useMemo. This is one of the most common causes of unnecessary re-renders in production React applications.

## Keys in Lists

When rendering a list of elements, React uses the 'key' prop to identify which items have changed, been added, or been removed. Using the array index as a key is discouraged when the list order can change, since it can cause React to misidentify elements and reuse the wrong internal state or DOM node.

# Section 5: SQL vs NoSQL Databases

Relational databases like PostgreSQL and MySQL store data in structured tables with predefined schemas, and enforce relationships between tables using foreign keys. NoSQL databases like MongoDB store data in flexible, often schema-less documents, which makes them easier to evolve but weaker on cross-record consistency guarantees.

## ACID Properties

ACID stands for Atomicity, Consistency, Isolation, and Durability, and describes a set of guarantees that traditional relational databases provide for transactions. Atomicity ensures a transaction either fully completes or has no effect at all, even if the system crashes partway through executing it.

## Indexing

A database index is a separate data structure, often a B-tree, that allows the database to find rows matching a query without scanning the entire table. Indexes speed up reads but slow down writes, since every insert or update must also update the index structure.

# Section 6: Git Version Control

Git is a distributed version control system where every developer has a full copy of the repository's history, unlike centralized systems where history lives only on a central server.

## Branching Strategy

A common branching strategy is trunk-based development, where developers merge small, frequent changes directly into the main branch rather than maintaining long-lived feature branches, which reduces the risk of large, painful merge conflicts.

## Rebase vs Merge

Git rebase rewrites commit history by moving a branch's commits onto a new base commit, producing a linear history, while git merge preserves the original commit history by creating a new merge commit. Rebasing shared/public branches is generally discouraged because it rewrites history other collaborators may depend on.